Uncorrelated Pseudo-Random Generator for FPGA

1st Thiago C. A. Paschoalin
Dept. Electroeletronic
Fed. Cent. for Technol. Educ. of Minas Gerais
Leopoldina, Brazil
https://orcid.org/0009-0008-4581-5285

2nd Ualison R. F. Dias
Elect. Eng. Postgraduate Program
Federal University of Juiz de Fora
Juiz de Fora, Brazil
https://orcid.org/0000-0002-0002-6812

3rd Melissa S. Aguiar
Elect. Eng. Postgraduate Program
Federal University of Juiz de Fora
Juiz de Fora, Brazil
https://orcid.org/0000-0002-5623-8128

4th Dabson F. dos Santos
Elect. Eng. Postgraduate Program
Federal University of Juiz de Fora
Juiz de Fora, Brazil
https://orcid.org/0009-0006-7233-4650

5th Tiago M. Quirino
Dept. Elect. Eng.
Rio de Janeiro State University
Rio de Janeiro, Brazil
https://orcid.org/0000-0002-1799-5222

6th Luciano M. de Andrade Filho
Dept. Elect. Eng.
Federal University of Juiz de Fora
Juiz de Fora, Brazil
https://orcid.org/0000-0003-1792-6793

2025 38th SBC/SBMicro/IEEE Symposium on Integrated Circuits and Systems Design (SBCCI) | 979-8-3315-5555-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/SBCCI66862.2025.11218676

***Abstract*—Random number is used in several applications including simulations, digital signal processing, machine learning and others. Concerning high-speed implementation, FPGAs are the most suitable technology. For this scenario, Linear Feedback Shift Register is a widely used approach, due to its parallel digital structure. However, the free-running sequence of the LFSR presents correlation among neighbor samples. This autocorrelated sequence makes the LSFR prohibitive for some applications. We propose a multi-core, parallel implementation, with different seeds, from which the final autocorrelation sequence is impulsive. Combining the output of several instances of the LSFR can overcome this limitation while maintaining the simplicity of the circuit. The FPGA implementation uses low resources of the board chosen, with a high rate of clock frequency.**

***Index Terms*—Random Number, FPGA, LFSR, Primitive Polynomials, Autocorrelation**

I. Introduction

The generation of random numbers is an important aspect in applications such as simulations, digital signal processing, statistical analysis, genetic algorithms, machine learning, and cryptography [1]–[3]. Additionally, random number generators can produce nanosecond-duration pulses emulating particle detectors, a specific application, as discussed in a previous work by Quirino et al. [4], for evaluating detector readout systems.

Hardware-based random number generation has become increasingly preferable over embedded software-based methods due to its best efficiency [5]. Random numbers are typically generated by two main classes: True Random Number Generators (TRNGs), which generate sequences where each number is completely independent of the previous ones, and Pseudo-Random Number Generators (PRNGs), which produce sequences that appear to be random but are actually determined by an initial seed value, meaning that they eventually repeat and can become predictable [6].

There are several ways to implement PRNGs, but the most common way is by using LFSR (Linear Feedback Shift Register), that is known for its velocity and can be easily implemented in FPGA (Field Programmable Gate Array) [7], being composed typically of a D flip-flop and XOR gates. The technique gives random numbers that are very difficult to predict or reproduce [8].

Since PRNG has repetitions and predictability, LFSR are implemented to increase the randomness of the PRNG with different techniques like polynomial modulators, which generate primitive polynomials with different bit lengths [9], [10]. It can also utilize a Resistance-Random-Access Memory Chip [11], or flying seeds that use the temperature oscillations [12], and many others.

In this work, it is proposed an implementation of a PRNG in FPGA that guarantees that the generated samples are not correlated with the adjacent ones by using seven LFSR generators with a selector that shifts the output number of the block. This achievement is made by using only XOR ports and a multiplexer.

The second section of this paper presents the theory behind Linear Feedback Shift Registers (LFSRs) used for pseudo-random number generation, highlighting the significance of primitive polynomials. Section III introduces the proposed circuit, designed for FPGA implementation, which generates a decorrelated sequence of samples. The fourth section details the FPGA board used and outlines the resource utilization. Finally, the last section provides the conclusions of the work.

II. LFSR Pseudo-Random Number Generator

The LFSR is a shift register that uses XOR logic gates for linear feedback, operating by shifting stored bits and calculating a new input bit based on a linear combination of previous bits, called the *feedback polynomial*. An n-bit LFSR consists of a shift register with n flip-flops and XOR gates connected to specific positions (called *taps*). The feedback bit is calculated as expressed in Eq. (1).

$$b_{\text{new}} = b_{k_1} \oplus b_{k_2} \oplus \cdots \oplus b_{k_m} \tag{1}$$

where b_{k_i} are the bits at positions defined by the feedback polynomial. The LFSR produces a bit sequence that repeats

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE JUIZ DE FORA. Downloaded on November 26,2025 at 20:30:34 UTC from IEEE Xplore. Restrictions apply.

after a maximum period of $2^n - 1$ cycles (if the polynomial is primitive). The new formed bit can be shifted in the Least Significant Bit (LSB) or in the Most Significant Bit (MSB) position.

A. LFSR Circuit with XOR Gates

An example of a 4-bit LFSR with feedback polynomial $x^4 + x^3 + 1$ (bits at positions 4 and 3) can be implemented with the circuit represented in the Figure 1.

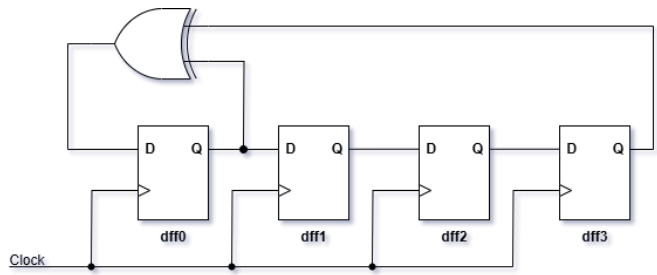

Fig. 1. 4-bit LFSR Circuit

In this way, we also have primitive polynomials, which are the mathematical foundation for LFSRs that generate maximum-length sequences. A polynomial P(x) of degree n is considered primitive if: (1) it is irreducible (cannot be factored into smaller polynomials), (2) its period is $2^n - 1$ (generating the maximum possible sequence), and (3) its roots are primitive elements in the Galois Field GF(2^n). When an LFSR uses a primitive polynomial, it will cycle through all possible $2^n - 1$ non-zero states before repeating, producing output sequences with excellent statistical properties, including balanced numbers of 0s and 1s, low correlation between consecutive bits, and uniform distribution of bit patterns.

The table I below shows examples of documented primitive polynomials.

TABLE I
PRIMITIVE POLYNOMIALS

Degree (n)	**Polynomial**	**Taps (hex)**
4	$x^4 + x + 1$	0x9
8	$x^8 + x^6 + x^5 + x^4 + 1$	0x1D
16	$x^{16} + x^{15} + x^{13} + x^4 + 1$	0xD008
32	$x^{32} + x^{22} + x^2 + x + 1$	0x400007

Recent advances in primitive polynomial research include GPU-accelerated search algorithms aiming to optimize exhaustive polynomial discovery through parallel computing [13], quantum-resistant constructions developing LFSR architectures resilient to Shor's algorithm attacks [14], and AI-optimized designs using machine learning to predict optimal polynomial configurations [15]. Standardization efforts like NIST SP 800-90B establish security criteria for post-quantum random number generation, while 3GPP Release 17 [16] specifically targets polynomial selection for 5G polar code error correction. Hardware security applications [17] focus on sparse polynomial implementations for tamper-resistant modules, with NIST FIPS 203 [18] providing updated cryptographic guidelines for government systems.

For practical implementations, engineers should prefer polynomials with few terms (reducing hardware complexity), avoid sparse polynomials in cryptographic applications, and use experimentally validated tables. As a concrete example, for $n = 3$, the primitive polynomial $x^3 + x + 1$ generates the sequence:

```
States: 111 → 011 → 101 → 010 → 001 →
→ 100 → 110 → 111...
Output: 1 1 1 0 1 0 0 (repeating)
```

showing the full period of $2^3 - 1 = 7$ non-zero states.

The polynomial choice directly impacts system design: more complex polynomials provide better security against attacks, polynomials with fewer taps offer faster hardware implementation, and the polynomial determines the output's statistical quality. While LFSRs are efficient, their linearity makes them vulnerable to attacks like the Berlekamp-Massey algorithm, leading modern cryptographic systems to often combine multiple LFSRs or use non-linear filtering for enhanced security.

III. PROPOSED CIRCUIT

A 42-bit LFSR PRNG is proposed using a primitive polynomial. This structure is chosen because it usually demands low hardware resources, and the number of bits generates a sequence running at 40 MHz that has a period of 1.27 days before repeating. The used primitive polynomial is represented in Eq. (2).

$$f(X) = X^{42} + X^{27} + X^{24} + X^{14} + X^8 + X^1 + 1 \quad (2)$$

Considering that the generated number has 42 bits, some applications do not require high output values. It is possible to use fewer bits, extracting the first LSBs (Least Significant Bits) that are needed. In this work, the 7 LSBs of the random number are used to do the analysis, but the results extend to higher-order bits. So, the circuit will generate random numbers between 0 and 127.

The first important analysis is the histogram of generated numbers. Considering that it has a uniform distribution, the flatter the histogram is, the better the result for the proposed technique. If a sequence of 3,000,000 samples is produced by the circuit, it reaches the histogram of Figure 2.

The histogram shows a flat pattern when generating sequences, even when it takes only the 7 LSBs. The autocorrelation of the sequence is another important characteristic since, in most of the applications, uncorrelated samples are desired. The plot obtained by this analysis must be an impulse in the center with the side values close to zero. The autocorrelation of the samples is represented in Figure 3.

At first glance, the samples seem to be uncorrelated, but when one takes the zoomed version of Figure 4, the first samples have some undesirable correlation. To avoid this problem, a solution is proposed.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE JUIZ DE FORA. Downloaded on November 26,2025 at 20:30:34 UTC from IEEE Xplore. Restrictions apply.

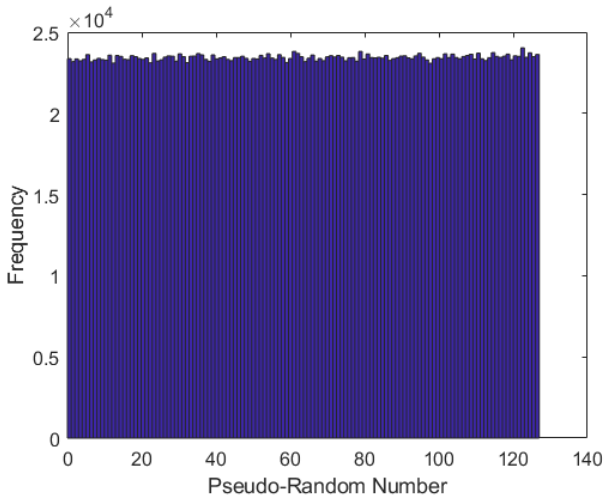

Fig. 2. Histogram of generated sequence of circuit

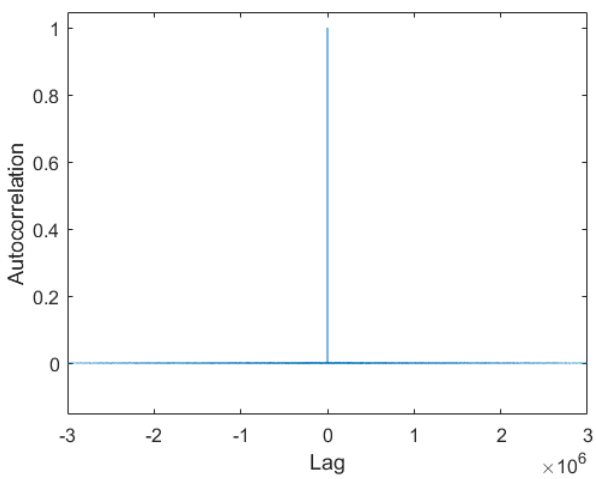

Fig. 3. Autocorrelation plot of generated sequence of circuit

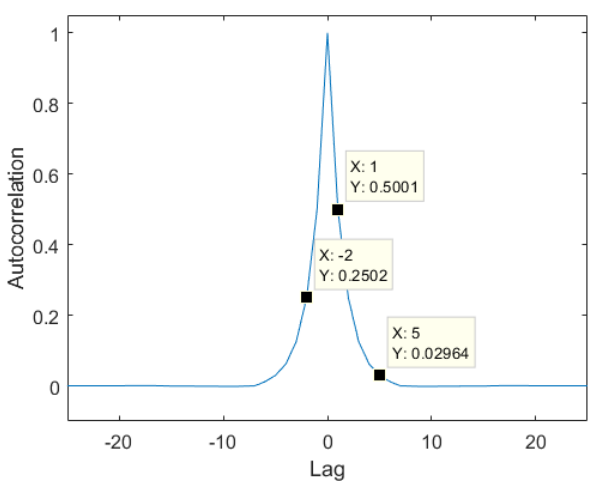

Fig. 4. Zoomed Autocorrelation plot of generated sequence of circuit

A. Parallel Structure

A solution for the correlation problem is to use multiple LFSR PRNGs (Pseudo-Random Number Generators) in parallel, producing different sequences (with different seeds) at the same frequency rate. A controlling circuit can be implemented to switch which LFSR PRNG output is going to be used in the next clock border. It is important that all PRNGs have distinguished seed values to accomplish the desired effect.

The number of parallel LFSR circuits depends on how the samples are correlated. Figure 4 shows that the fifth sequenced sample still exhibits a minimal correlation. In this paper, seven structures are used to ensure very low correlation. Depending on the target application, this number can be either decreased or increased.

The proposed solution is illustrated in Figure 5. A counter, with a modulus equal to the number of parallel LFSR circuits, selects which one will be used. This approach allows the initial undesired correlated samples from each structure to be discarded.

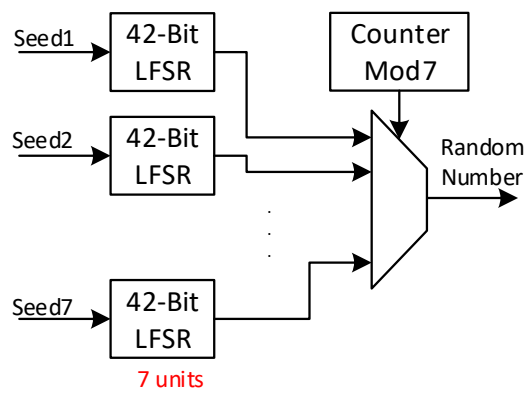

Fig. 5. Proposed solution

The output random number has to maintain the same flat distribution of the histogram, and it is shown in Figure 6 that this characteristic remains. The new autocorrelation, zoomed at the problematic region, is represented in Figure 7. It is possible to verify that now the sequence has uncorrelated samples, achieving the desired pattern.

IV. FPGA Implementation

The project target chosen to test is the development kit designed by Terasic, the DE0-Nano-SoC. This board uses a Cyclone V FPGA family manufactured by Intel.

The same structure shown in Figure 5 was implemented in FPGA, with 7 output bits. The table II synthesizes the resource usage of the device and shows the maximum clock frequency allowed using the proposed solution.

The PRNG uses few resources of the board. Less than 1% of ALMs inside the Cyclone V were required. The null DSP

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE JUIZ DE FORA. Downloaded on November 26,2025 at 20:30:34 UTC from IEEE Xplore. Restrictions apply.

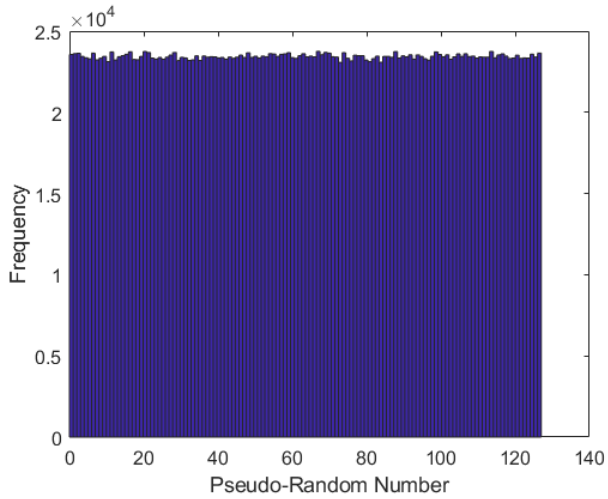

Fig. 6. Zoomed Autocorrelation plot of the new generated sequence of circuit

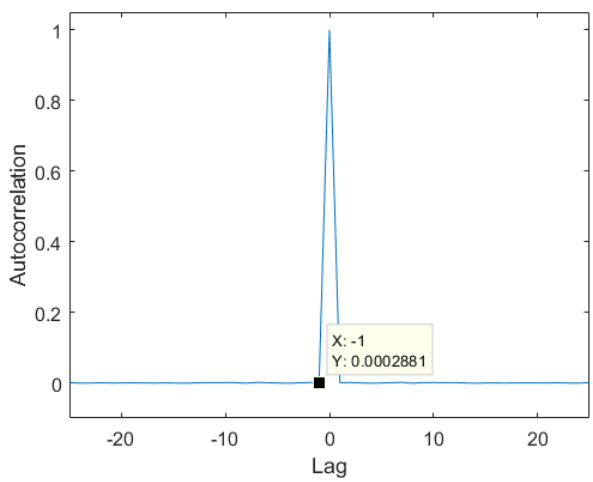

Fig. 7. Zoomed Autocorrelation plot of the new generated sequence of circuit

TABLE II
RESOURCES USAGE OF DE0-NANO-SOC

Resource	Value	Percentage from Total
Logic Utilization (ALMs)	87	$< 1\%$
Total Registers	323	-
Total pins	9	3%
Total memory blocks bits	0	0%
Total DSP Blocks	0	0%
Maximum Frequency	381.68 MHz	-

block usage saves this precious resource for other complex processes that may exist. Another great advantage is the maximum frequency of the clock. High-speed applications can be done with this structure.

V. CONCLUSION

The LFSR circuit is an efficient solution to generate pseudo-random numbers for hardware applications. It is easy to implement in FPGA since it uses logic gates and bit shift operations. The LFSR PRNG is able to produce a sequence of numbers with uniformly distributed samples. However, the samples are correlated. The solution of using more parallel LFSR structures with a multiplexer to generate the random sequence has corrected this mentioned issue without increasing the complexity of the circuit. The implementation uses less than 1% of the logic provided by the Cyclone V FPGA family used, and it can run in a high-speed clock.

REFERENCES

[1] R. F. W. Coates, G. J. Janacek and K. V. Lever, "Monte Carlo simulation and random number generation," in IEEE Journal on Selected Areas in Communications, vol. 6, no. 1, pp. 58-66, Jan. 1988, doi: 10.1109/49.192730.

[2] Y. Wen and W. Yu, "Machine learning-resistant pseudo-random number generator". Electron. Lett., 55: 515-517, 2019, doi: 10.1049/el.2019.0485

[3] V. L. R. d. Costa, J. López and M.V. Ribeiro, "A System-on-a-Chip Implementation of a Post-Quantum Cryptography Scheme for Smart Meter Data Communications". Sensors, vol. 22, no. 19, 2022, doi: 10.3390/s22197214

[4] T. M. Quirino, T. C. A. Paschoalin, G. I. Gonçalves, P. H. B. Lisboa, L. M. de Andrade Filho, and B. S.-M. Peralva, "Online Pulse Compensation for Energy Spectrum Determination: A Pole-Zero Cancellation and Unfolding Approach", Electronics, vol. 14, no. 3, 2025, doi: 10.3390/electronics14030493.

[5] Y. Li, P. Chow, J. Jiang, M. Zhang, and S. Wei, "Software/Hardware Parallel Long-Period Random Number Generation Framework Based on the WELL Method," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 22, no. 5, pp. 1054-1059, May 2014.

[6] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, 3rd ed. Reading, MA: Addison-Wesley, 1998.

[7] G. Madhupavani and K. Suneetha, "Design of Random Number Generation Using 256 Bit LFSR in FPGA," *International Journal of Advanced Technology and Innovative Research*, vol. 9, pp.

[8] B. Krishna, G. L. Murali, M. Madhumati, and H. Khan, "FPGA based pseudo random sequence generator using XOR/XNOR for communication cryptography and VLSI testing applications," *International Journal of Innovative Technology and Exploring Engineering*, vol. 8, no. 4, pp. 485-494, 2019.

[9] R. S. Durga, C. Rashmika, O. N. Madhumitha, D. Suvetha, B. Tanmai, and N. Mohankumar, "Design and synthesis of LFSR based random number generator," in *2020 Third International Conference on Smart Systems and Inventive Technology (IC-SSIT)*, pp. 438–442, IEEE, 2020.

[10] K. Bailey et al., "FPGA implementation of reversible LFSR with primitive polynomial using Verilog HDL," in *2022 IEEE International Conference on Distributed Computing and Electrical Circuits and Electronics (ICDCECE)*, pp. 1–5, IEEE, 2022.

[11] V. V. Gudla and V. S. S. S. S. Jyothi, "Design and implementation of digital clock manager based pseudo-true random number generator," in *2022 IEEE 3rd Global Conference for Advancement in Technology (GCAT)*, pp. 1–5, IEEE, 2022.

[12] P. Zode, P. Zode, and R. Deshmukh, "FPGA based novel true random number generator using LFSR with dynamic seed," *2019 IEEE 16th India Council International Conference (INDICON)*, pp. 1-3, 2019.

[13] G. Borowik and A. Paszkiewicz, "Hardware Accelerator for Generating Primitive Polynomials over GF(3)," *2011 21st International Conference on Systems Engineering*, Las Vegas, NV, USA, 2011, pp. 486-487, doi: 10.1109/ICSEng.2011.99.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE JUIZ DE FORA. Downloaded on November 26,2025 at 20:30:34 UTC from IEEE Xplore. Restrictions apply.

[14] Hosoyamada2024, Akinori. ”Quantum algorithms for fast correlation attacks on lfsr-based stream ciphers.” *International Conference on the Theory and Application of Cryptology and Information Security*. Singapore: Springer Nature Singapore, 2024.
[15] Abu-Khader, Nabil. Arithmetic operations in finite fields using multiple-valued logic. Wayne State University, 2005.
[16] 3GPP Technical Specification 38.212, ”NR; Multiplexing and channel coding (Release 17),” v17.1.0, Mar. 2022.
[17] Narayanan, S., Raghunathan, A., Venkatesan, R.: Obfuscating straight line arithmetic programs. *In: Proceedings of the Nineth ACM Workshop on Digital Rights Management*. DRM 2009, pp. 47–58. ACM, New York 2009
[18] NIST FIPS 203, ”Module-Lattice-Based Key-Encapsulation Mechanism Standard,” Aug. 2023.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE JUIZ DE FORA. Downloaded on November 26,2025 at 20:30:34 UTC from IEEE Xplore. Restrictions apply.